

SETUP & STARTUP GUIDE

PMI Reporting Agent

Windows installation on a new device

VS Code • Git • Python • Node.js • read-only GitHub access • API keys

Outcome: Follow the steps in order. At the end, the agent opens locally at <http://localhost:8000>.

5 August 2026

1. Before you begin

This guide assumes a normal 64-bit Windows 10 or Windows 11 computer. Docker and Windows Subsystem for Linux are not required for this local setup.

What you need

1. A Windows account that is allowed to install applications.
2. A GitHub account. Give the project owner the exact GitHub username before starting.
3. An invitation granting read access to the private repository, accepted in the same GitHub account used for cloning.
4. Internet access for installers and dependency downloads.
5. Optional: an OpenAI API key, an Anthropic API key, or both. The app runs without a key, but its semantic summaries are limited and it cannot interpret images.

Read-only access: The user may clone and pull code. GitHub rejects pushes when the repository role has no write permission. The user can still create local files and local edits on their own computer.

Install order at a glance

6. **Install VS Code** — the editor and terminal used in this guide
7. **Install Git for Windows** — includes secure browser-based GitHub sign-in
8. **Install Python 3.13 (64-bit)** — the application requires Python 3.12 or newer
9. **Install Node.js LTS** — needed to build the React frontend
10. **Restart VS Code** — so the new PATH entries are visible
11. **Clone, configure, build, and start** — all commands are provided below

2. Install the required software

Download installers only from the official sources below. Choose x64 unless the Windows device is specifically ARM-based.

Tool	Install choice	Official source
VS Code	Windows User Setup	VS Code Windows setup
Git	Git for Windows x64	Git for Windows
Python	Python 3.13, 64-bit	Python for Windows
Node.js	Current LTS, Windows x64	Node.js downloads

2.1 Visual Studio Code

12. **Open the official VS Code Windows setup page** — use the Windows User Setup installer for most users.
13. **Run the downloaded installer** — accept the licence and default install location.
14. **Keep “Add to PATH” enabled** — also enable “Open with Code” options if offered.
15. **Finish the install** — do not run VS Code as Administrator for normal use.

Official page: <https://code.visualstudio.com/docs/setup/windows>

2.2 Git for Windows

16. **Open the official Git for Windows page** — download the current 64-bit installer.
17. **Run the installer** — the default options are suitable for this project.
18. **Keep Git Credential Manager installed** — it enables secure GitHub authentication through the browser and supports two-factor authentication.

Official page: <https://git-scm.com/install/windows>

2.3 Python

19. **Open Python Releases for Windows** — choose a supported Python 3.13 64-bit installer. The project accepts Python 3.12 or newer; 3.13 is the recommended compatibility choice for this guide.
20. **Run the installer** — if the installer offers “Add Python to PATH”, enable it.
21. **Complete the installation** — the Windows Python launcher command used below is py -3.13.

Official page: <https://www.python.org/downloads/windows/>

2.4 Node.js

22. **Open the Node.js download page** — select the current LTS release for Windows x64.
23. **Run the .msi installer** — keep npm and “Add to PATH” selected.
24. **Finish the install** — extra native build tools are not required for this project’s normal installation.

Official page: <https://nodejs.org/en/download>

2.5 Restart and verify

Close every open VS Code and PowerShell window, then reopen VS Code. Select Terminal > New Terminal. The terminal type shown in the top-right should be PowerShell.

RUN ONE LINE AT A TIME

```
git --version
py -3.13 --version
node --version
npm.cmd --version
code --version
```

Expected result: Every command prints a version number. If a command is not recognised, restart Windows once. If it still fails, reinstall that tool and ensure its PATH option is enabled.

Optional VS Code extension

In VS Code, open Extensions (Ctrl+Shift+X), search for “Python”, and install the extension published by Microsoft. It is helpful for viewing the code, but the terminal commands work without it.

3. Get access and clone the code

3.1 Project owner: provides grant pull-only access

Project Owner invites the recipient's GitHub username to the repository with a role that can read/clone but cannot write.

GitHub Link is in the separate file or ask project owner (Sören Lange-Klapproth) for the invitation.

3.2 Recipient: accept the invitation

25. **Sign in to GitHub** — use the exact account that was invited.
26. **Open the invitation** — use the GitHub notification or invitation email and select Accept invitation.
27. **Open the repository page** — confirm that the files are visible before continuing.

3.3 Clone in the VS Code terminal

In VS Code, select Terminal > New Terminal. Paste each line below and press Enter. The first command chooses the Windows Documents folder; the second downloads the repository; the third enters it; the fourth opens it in VS Code.

POWERSHELL

```
Set-Location "$env:USERPROFILE\Documents"  
git clone https://github.com/SoerenM2345/PMI-Reporting-Agent.git  
Set-Location .\PMI-Reporting-Agent  
code .
```

GitHub sign-in: For a private repository, Git Credential Manager normally opens a browser. Sign in, authorise the request if prompted, and complete two-factor authentication. Do not place a password or token inside the clone URL.

When VS Code asks whether you trust the folder, confirm only if the repository URL and owner match the project details above.

3.4 Confirm the clone

POWERSHELL

```
git remote -v  
git branch --show-current  
git status --short
```

28. The remote should point to <https://github.com/SoerenM2345/PMI-Reporting-Agent.git>.
29. The initial branch should normally be main.

30. `git status --short` should print nothing on a new clone.

GitHub cloning guide: <https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository>

GitHub credential guide: <https://docs.github.com/en/get-started/git-basics/caching-your-github-credentials-in-git>

4. Install the project dependencies

Location check: Run every command in the repository root—the folder containing README.md, requirements.txt, .env.example, app, and frontend.

The setup uses a project-local Python virtual environment named .venv. Commands call its Python executable directly, so PowerShell execution-policy changes and activation steps are unnecessary.

4.1 Create the Python environment

POWERSHELL

```
py -3.13 -m venv .venv
```

4.2 Install Python packages

POWERSHELL

```
.\.venv\Scripts\python.exe -m pip install --upgrade pip  
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

The first installation downloads FastAPI, reporting libraries, document readers, and both supported model-provider clients. It may take several minutes.

4.3 Install and build the frontend

POWERSHELL

```
npm.cmd --prefix frontend ci  
npm.cmd --prefix frontend run build
```

The build must finish successfully because FastAPI serves frontend/dist. npm.cmd is used deliberately to avoid the common PowerShell “npm.ps1 cannot be loaded” execution-policy error.

Success indicators: The Python install ends without a red ERROR. The frontend build reports that files were written to frontend/dist.

5. Add your own API key(s)

API keys are optional but recommended. Without a key, deterministic extraction, consistency checks, conflict resolution, and file generation still run. Semantic summaries use fallback prose and images cannot be interpreted.

Correct file: Store secrets only in `PMI-Reporting-Agent\.env` at the repository root. Do not put keys in `app/config.py`, frontend files, `README.md`, or any file ending in `.example`.

5.1 Create the private .env file

POWERSHELL

```
Copy-Item .env.example .env
code .env
```

The first command copies the template. The second opens the new private file in VS Code. Windows may hide files whose names start with a dot, but VS Code shows `.env` in the Explorer panel.

CORRECT LOCATION

```
PMI-Reporting-Agent\
.env           ← put real keys here; private and Git-ignored
.env.example   ← template only; never put real keys here
app\config.py  ← model catalogue; do not put secrets here
frontend\
```

5.2 Choose one provider configuration

In `.env`, replace the `provider/key/model` lines with one of the blocks below. Do not leave the template values `sk-...` or `sk-ant-...` in active lines. Do not add quotation marks or spaces around the equals sign.

Option A — OpenAI only

CONTENTS OF .ENV

```
# ANTHROPIC_API_KEY=
OPENAI_API_KEY=PASTE_YOUR_REAL_OPENAI_KEY_HERE
LLM_PROVIDER=openai
LLM_MODEL=gpt-5.6
VISION_MODEL=gpt-5.6
FAST_MODEL=gpt-4o-mini
```

Option B — Anthropic only

CONTENTS OF .ENV

```
ANTHROPIC_API_KEY=PASTE_YOUR_REAL_ANTHROPIC_KEY_HERE
# OPENAI_API_KEY=
LLM_PROVIDER=anthropic
LLM_MODEL=claude-opus-4-8
VISION_MODEL=claude-sonnet-5
FAST_MODEL=claude-haiku-4-5
```

Option C — both providers

Keep both real key lines active. Choose the default provider with LLM_PROVIDER and make LLM_MODEL, VISION_MODEL, and FAST_MODEL match that default provider using Option A or B. The user interface can still switch individual chats to the other configured provider.

Why model lines matter: The copied template uses Anthropic model names. If LLM_PROVIDER is changed to openai, the three model lines must also be changed to the OpenAI values shown above.

5.3 Save and verify without revealing the key

POWERSHELL

```
git check-ignore .env
.\.venv\Scripts\python.exe -c "from app.config import get_settings; s=get_settings();
print('provider=', s.llm_provider, 'configured=', s.llm_configured())"
```

31. git check-ignore .env should print .env. This confirms Git will not track it.
32. The configuration check should print the selected provider and configured= True.
33. This check does not make a paid model request and does not print the secret.

OpenAI API key settings: <https://platform.openai.com/api-keys> — create a normal project API key, not an organisation admin key

Anthropic Console: <https://console.anthropic.com/> — create a key in the API Console

6. Start, use, and stop the agent

6.1 Start the local server

Make sure the terminal is still in the repository root. Start FastAPI with the project's virtual-environment Python:

POWERSHELL

```
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload
```

Keep the terminal open: The server runs only while this command is active. Do not close that terminal window while using the agent.

6.2 Open the agent

34. **Open a web browser** — Microsoft Edge, Chrome, or Firefox is suitable.
35. **Go to <http://localhost:8000>** — localhost means this Windows computer; it is not a public website.
36. **Confirm the interface loads** — create or open a project, then use the model picker to confirm the intended provider is available.
37. **Use only suitable test files first** — API calls can incur provider charges, especially for images or large file sets.

Local application: <http://localhost:8000>

Local API documentation: <http://localhost:8000/docs> — optional technical check while the server is running

6.3 Stop the agent

Return to the terminal that is running Uvicorn and press Ctrl+C. Wait until the prompt returns. Closing the browser tab alone does not stop the server.

6.4 Start it again later

POWERSHELL

```
Set-Location "$env:USERPROFILE\Documents\PMI-Reporting-Agent"
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload
```

There is no need to recreate .venv, reinstall packages, rebuild the frontend, or recreate .env every time. Repeat those tasks only after an update requires them or if the local environment was removed.

7. Pull code updates safely

Pull-only access is sufficient for receiving project updates. The private .env file, generated output, uploaded working data, .venv, frontend/node_modules, and frontend/dist are excluded from Git tracking and are not pushed.

Recommended update workflow

38. **Stop the server** — press Ctrl+C in its terminal.
39. **Check for local code edits** — `git status --short` should normally be empty.
40. **Pull the latest main branch** — the `--ff-only` option avoids creating an accidental merge commit.
41. **Refresh dependencies if needed** — rerunning the install commands is safe when unsure.
42. **Rebuild the frontend** — `frontend/dist` is local and is not stored in Git.
43. **Restart the server** — use the normal Uvicorn command.

CONSERVATIVE UPDATE AND START

```
Set-Location "$env:USERPROFILE\Documents\PMI-Reporting-Agent"
git status --short
git pull --ff-only
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
npm.cmd --prefix frontend ci
npm.cmd --prefix frontend run build
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload
```

If git status shows files: Do not delete or overwrite them blindly. They are local changes. Ask the project owner whether to keep them before pulling. The .env file should not appear because it is Git-ignored.

8. Troubleshooting

“git”, “py”, “node”, “npm”, or “code” is not recognised

44. Close all terminals and VS Code, then reopen them. If needed, restart Windows.
45. Run the version checks from section 2.5 again.
46. Reinstall only the missing tool and ensure its PATH option is enabled.

Repository not found or HTTP 403

47. Confirm the GitHub invitation was accepted.
48. Confirm the browser is signed in to the invited GitHub account, not a different personal or work account.
49. Open the repository URL in the browser. If it is not visible there, the repository owner must correct access.
50. If Windows cached the wrong GitHub account, open Control Panel > Credential Manager > Windows Credentials and remove the relevant github.com entry, then clone again and sign in with the correct account.

PowerShell says npm.ps1 cannot be loaded

Use npm.cmd exactly as shown in this guide. It calls npm without changing the computer’s execution policy.

Python package installation fails

51. Confirm `py -3.13 --version` works and shows Python 3.13.x.
52. Confirm the terminal is in the repository root and `requirements.txt` is visible.
53. On a managed corporate device, a proxy, antivirus product, or missing certificate may block downloads; contact IT rather than disabling security controls.

The browser shows a 404, blank page, or frontend build warning

REBUILD FRONTEND, THEN RESTART UVICORN

```
npm.cmd --prefix frontend ci
npm.cmd --prefix frontend run build
```

The API provider is unavailable or configured=False

54. Confirm the file is named exactly `.env`, not `.env.txt`.

55. Confirm .env is beside README.md and requirements.txt.
56. Confirm LLM_PROVIDER matches the key and model block from section 5.
57. Remove placeholder keys, save the file, stop Uvicorn, and start it again.

Port 8000 is already in use

Another copy may already be running. Stop it with Ctrl+C in its terminal. If that terminal is unavailable, start temporarily on another port:

POWERSHELL

```
| .\.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8001
```

Then open <http://localhost:8001>.

9. Security and handover checklist

API keys are secrets: They may create paid usage. Never paste them into chat, email, screenshots, tickets, presentations, source code, or Git. If a key is exposed, revoke it immediately in the provider console and create a replacement.

Recipient checklist

58. GitHub invitation accepted with the intended account.
59. VS Code, Git, Python 3.13, and Node.js LTS installed and version-checked.
60. Repository cloned from the approved GitHub URL.
61. .venv created and Python dependencies installed.
62. Frontend dependencies installed and frontend/dist built.
63. Private .env created at the repository root.
64. Unused placeholder API key lines removed or commented out.
65. git check-ignore .env confirms the secret file is ignored.
66. Configuration check prints the chosen provider and configured=True, if using an API.
67. Agent opens at <http://localhost:8000>
68. Recipient understands that git pull is allowed and git push is not.

10. Official references

These links are included for current installer and authentication guidance. Project-specific commands and configuration in this guide were verified against this repository's README.md, .env.example, app/config.py, requirements.txt, pyproject.toml, and frontend/package.json on 5 August 2026.

VS Code on Windows: <https://code.visualstudio.com/docs/setup/windows> — official Microsoft setup guide

Git for Windows: <https://git-scm.com/install/windows> — official Git download

Python for Windows: <https://www.python.org/downloads/windows/> — official Python releases

Node.js downloads: <https://nodejs.org/en/download> — choose the LTS Windows installer

Clone a GitHub repository: <https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository> — official GitHub steps

Git Credential Manager on Windows: <https://docs.github.com/en/get-started/git-basics/caching-your-github-credentials-in-git> — official browser sign-in guidance

OpenAI API authentication: <https://platform.openai.com/docs/api-reference/authentication> — official reminder to keep API keys secret

Anthropic API access: <https://support.anthropic.com/en/articles/8114521-how-can-i-access-the-anthropic-api> — official Console/key guidance